

Relational algebra

Objectives

- How to form queries in relational algebra

Introduction

- Similar to normal algebra (as in $2+3*x-y$), except we use relations as values instead of numbers, and the operations and operators are different.
- Not used as a query language in actual DBMSs. (SQL instead.)
- The inner, lower-level operations of a relational DBMS are, or are similar to, relational algebra operations. We need to know about relational algebra to understand query execution and optimization in a relational DBMS.
- Some advanced SQL queries requires explicit relational algebra operations, most commonly *outer join*.

- Relations are seen as *sets of tuples*, which means that no duplicates are allowed. SQL behaves differently in some cases. Remember the SQL keyword `distinct`.
- SQL is *declarative*, which means that you tell the DBMS *what* you want, but not *how* it is to be calculated. A C++ or Java program is *procedural*, which means that you have to state, step by step, exactly how the result should be calculated. Relational algebra is (more) procedural than SQL. (Actually, relational algebra is mathematical expressions.)

Relational Algebra

- Relational algebra is high level procedural language
- Relational algebra operations work on one or more relations to define another relation without changing original relations
- Both operands and results are relations
 - Output from one relation can be input to another
- Allows nested expressions
 - Closure property

Review of concepts and operations from set theory

- set
- element
- no duplicate elements (but: multiset = bag)
- no order among the elements (but: ordered set)
- subset
- proper subset (with fewer elements)
- superset
- union
- intersection
- set difference
- cartesian product

Relational Algebra

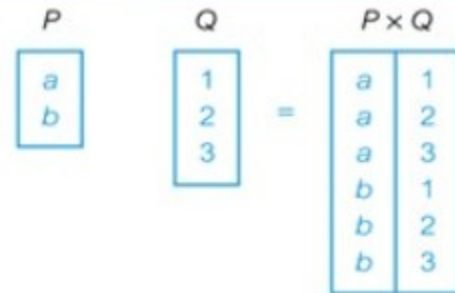
- Basic operations
 - **Selection, Projection, Union, Set difference, Cartesian product and Rename**
- Perform most required data retrieval operations
- Additional operations
 - **join, set intersection, Division and assignment**
 - Can be expressed in terms of the basic operations.



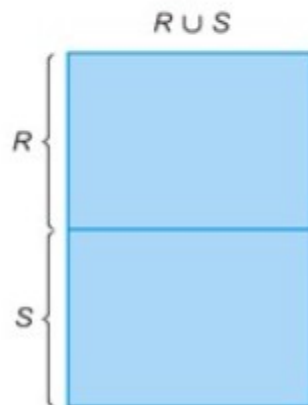
(a) Selection



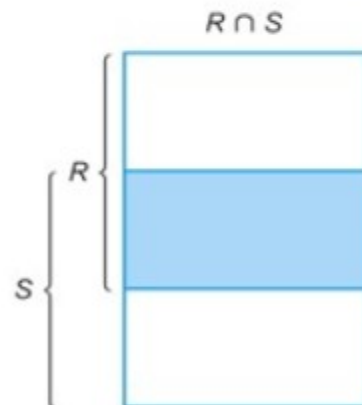
(b) Projection



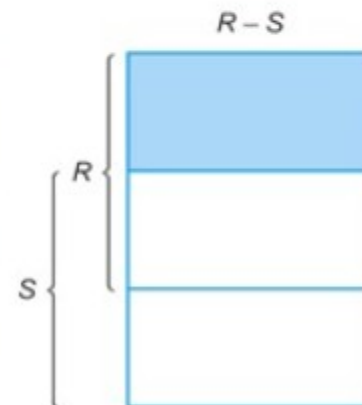
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

T	
A	B
a	1
b	2

U	
B	C
1	x
1	y
3	z

$T \bowtie U$		
A	B	C
a	1	x
a	1	y

$T \triangleright_B U$	
A	B
a	1

$T \bowtie_c U$		
A	B	C
a	1	x
a	1	y
b	2	

(g) Natural join

(h) Semijoin

(i) Left Outer join

R	
	Remainder

S

$R + S$

V	
A	B
a	1
a	2
b	1
b	2
c	1

W
B
1
2

$V + W$
A
a
b

(j) Divis on (shaded area)

Example of division

Operation	My HTML	Symbol
Projection	PROJECT	π
Selection	SELECT	σ
Renaming	RENAME	ρ
Union	UNION	$\cup$
Intersection	INTERSECTION	$\cap$
Assignment	$\leftarrow$	$\leftarrow$

Operation	My HTML	Symbol
Cartesian product	X	$\times$
Join	JOIN	$\bowtie$
Left outer join	LEFT OUTER JOIN	$\bowtie_{\text{L}}$
Right outer join	RIGHT OUTER JOIN	$\bowtie_{\text{R}}$
Full outer join	FULL OUTER JOIN	$\bowtie_{\text{F}}$
Semijoin	SEMIJOIN	$\bowtie_{\text{S}}$

Basic Operations

- **Unary** operators: operate on one relation
 - Selection, Projection, Rename
- **Binary** operators: operate on pairs of relations
 - Union, Set Difference, Cartesian product
- The result of an operation is a new relation.

Selection(or Restriction)

- We use the Greek letter sigma (σ) for the operator.

$\sigma_{\text{predicate}} (R)$

- Works on single relation R
- Defines relation that contains only tuples (rows) of R that satisfy specified condition (*predicate*)

- The predicate has a comparison operator ($>$, $<$, $=$, $\neq$ (not equal), $\geq$, $\leq$).
- You can compare values in different attributes or compare an attribute to a constant value.
- It can also include logical AND, OR and NOT operators: $\wedge$, $\vee$, $\neg$.

Example –Selection(or restriction)

List all staff with a salary greater than £10,000.

$\sigma_{\text{salary} > 10000}$ (Staff)

<u>StaffNO</u>	<u>fName</u>	<u>lName</u>	Position	sex	DOB	Salary	branchNo
SL21	Henok	Getachew	Manager	M	1-oct-45	30,000	B005
SG37	Saron	Tadesse	Assistant	F	10-nov-60	12,000	B003
SG45	Mekonnen	Guta	Supervisor	M	24-mar -50	18,000	B003
SG60	Adey	Goytom	Manager	F	16-jun-40	24,000	B003

The same table **E** (for **EMPLOYEE**) as above.

SQL	Result	Relational algebra									
<pre>select * from E where salary < 200</pre>	<table> <tr> <th>nr</th><th>name</th><th>salary</th></tr> <tr> <td>1</td><td>John</td><td>100</td></tr> <tr> <td>7</td><td>Tom</td><td>100</td></tr> </table>	nr	name	salary	1	John	100	7	Tom	100	$\text{SELECT}_{\text{salary} < 200}(\text{E})$
nr	name	salary									
1	John	100									
7	Tom	100									
<pre>select * from E where salary < 200 and nr >= 7</pre>	<table> <tr> <th>nr</th><th>name</th><th>salary</th></tr> <tr> <td>7</td><td>Tom</td><td>100</td></tr> </table>	nr	name	salary	7	Tom	100	$\text{SELECT}_{\text{salary} < 200 \text{ and nr} \geq 7}(\text{E})$			
nr	name	salary									
7	Tom	100									

Projection

We use the greek letter Pi for the operand (Π)

$\Pi_{\text{col1}, \dots, \text{coln}}(\mathbf{R})$

- Works on a single relation $\mathbf{R}$
- Defines relation that contains vertical subset of $\mathbf{R}$
- Extracts values of specified attributes
- Eliminates duplicates

Example- Projection

Produce a list of salaries for all staff, showing only staffNo, fName, lName, and salary details.

$\Pi_{\text{staffNo, fName, lName, salary}} (\text{Staff})$

<u>StaffNO</u>	<u>fName</u>	<u>lName</u>	Salary
SL21	Henok	Getachew	30,000
SG37	Saron	Tadesse	12,000
SG45	Mekonnen	Guta	18,000
SG60	Adey	Goytom	24,000
SG7	Hana	Belete	6,000

<u>StaffNO</u>	<u>fName</u>	<u>lName</u>	Position	sex	DOB	Salary	branchNo
SL21	Henok	Getachew	Manager	M	1-oct-45	30,000	B005
SG37	Saron	Tadesse	Assistant	F	10-nov-60	12,000	B003
SG45	Mekonnen	Guta	Supervisor	M	24-mar -50	18,000	B003
SG60	Adey	Goytom	Manager	F	16-jun-40	24,000	B003
SG7	Hana	Belete	Secretary	F	17-may-55	6,000	B005

Selection (yellow)

Projection (red)

Example: The table **E** (for **EMPLOYEE**)

nr	name	salary
1	John	100
5	Sarah	300
7	Tom	100

SQL	Result	Relational algebra								
<pre>select salary from E</pre>	<table><tr><th>salary</th></tr><tr><td>100</td></tr><tr><td>300</td></tr></table>	salary	100	300	$\text{PROJECT}_{\text{salary}}(\text{E})$					
salary										
100										
300										
<pre>select nr, salary from E</pre>	<table><tr><th>nr</th><th>salary</th></tr><tr><td>1</td><td>100</td></tr><tr><td>5</td><td>300</td></tr><tr><td>7</td><td>100</td></tr></table>	nr	salary	1	100	5	300	7	100	$\text{PROJECT}_{\text{nr, salary}}(\text{E})$
nr	salary									
1	100									
5	300									
7	100									

Note that there are no duplicate rows in the result.

How selection and projection are used in SQL ?

Select *

From Staff

Where Salary >10,000;

- What part is equivalent to a relational algebra select operation – can you see an argument relation and a predicate?
- What part is equivalent to the project operation?

SELECT staffNO, fName,lName,Salary

From Staff;

Union

$R \cup S$

- Defines relation that contains all tuples of R, or S, or both R and S
- Duplicate tuples eliminated

When doing a union operation:

Must use **compatible** relations – it must make sense to union the 2 relations. For example, it does not make sense to union the student and course relations.

The relations being union-ed must have the same number of attributes – we say that the relations have the same *arity*.

The domains of the corresponding attributes must be the same. In other words: for $r \cup s$, the domain of the i^{th} attribute of r must be the same domain as that for the i^{th} attribute of s .

For relations R and S with I and J tuples, respectively

- Union is concatenation of R & S into one relation with maximum of $(I + J)$ tuples

Exercise- Union

- list of all student and teacher names, along with their email addresses

Teacher-Schema (teacherid, teacher_firstname, teacher_fathersname, teacher_email)

Student-schema = (student_id, student_firstname, student_fathersname, programme_code, student_email)

$\Pi_{\text{student_firstname, student_fathersname, student_email}}(\text{student})$

$\cup$

$\Pi_{\text{teacher_firstname, teacher_fathersname, teacher_email}}(\text{teacher})$

Set-Difference

$R - S$

- Defines relation consisting of tuples in relation R , but not in S
- R and S must be union-compatible

Example-Set difference

- find students who are taking the ICT252 course but who are not taking the ICT231 course ?

- get a relation showing students taking ICT252 – just get the student ID

$$\Pi_{\text{student_id}} (\sigma_{\text{course_code}=\text{"ICT252"}} (\text{studentCourse}))$$

- get a relation showing students taking ICT231 – should be compatible with the first relation:

$$\Pi_{\text{student_id}} (\sigma_{\text{course_code}=\text{"ICT231"}} (\text{studentCourse}))$$

- Now we can get the difference between these two:

$$\Pi_{\text{student_id}} (\sigma_{\text{course_code}=\text{"ICT252"}} (\text{studentCourse})) -$$

$$\Pi_{\text{student_id}} (\sigma_{\text{course_code}=\text{"ICT231"}} (\text{studentCourse}))$$

Cartesian Product

$R \times S$

- **Binary operation**
- **Defines relation that is concatenation of every tuple of relation R with every tuple of relation S**
- The result is a relation that has all the attributes from both relations and whose tuples are all the possible combinations of the tuples from each of the 2 relations.
- so if we have n tuples in R and m tuples in S , $r(R \times S)$ contains $(n \times m)$ tuples.

Sample data

Student

student_ID	student_firstname	student_fathersname	Program_code
100	Sara	Tadesse	CSDEG
101	Tekle	Desta	CSDEG
102	Abebe	Girma	CSDEG
103	Solomon	Kebede	CSDIP

Programme

programme_code	prog_name	prog_description
CSDEG	Computer Science Degree	3 Year Degree in Computer Science
CSDIP	Computer Science Diploma	Diploma in Computer Science

Example-Cartesian Product

`r=Student X Programme`

- The attributes in `r` are:
(`student.stud_id`, `student.stud_firstname`,
`student.stud_fathersname`,
`student.prog_code`, `programme.prog_code`,
`programme.prog_name`,
`programme.prog_desc`)

student_ID	student_firstname	student_fathersname	Program_code	programme_code	prog_name	prog_description
100	Sara	Tadesse	CSDEG	CSDEG	Computer Science Degree	3 Year Degree in Computer Science
100	Sara	Tadesse	CSDEG	CSDIP	Computer Science Diploma	Diploma in Computer Science
101	Tekle	Desta	CSDEG	CSDIP	Computer Science Diploma	Diploma in Computer Science
101	Tekle	Desta	CSDEG	CSDEG	Computer Science Degree	3 Year Degree in Computer Science
102	Abebe	Girma	CSDEG	CSDIP	Computer Science Diploma	Diploma in Computer Science
102	Abebe	Girma	CSDEG	CSDEG	Computer Science Degree	3 Year Degree in Computer Science
103	Solomon	Kebede	CSDIP	CSDIP	Computer Science Diploma	Diploma in Computer Science
103	Solomon	Kebede	CSDIP	CSDEG	Computer Science	3 Year Degree in Computer

Example-on Cartesian product, selection and projection

Get name of all students on the Computer Science Degree programme ?

$$R = \Pi_{\text{stud_firstname}, \text{stud_fathersname}} \left(\sigma_{\text{student.prog_code} = \text{programme.prog_code}} \left(\sigma_{\text{prog_name} = \text{"Computer Science Degree"}} (\text{student} \times \text{programme}) \right) \right)$$

- Cartesian product and selection can be used to reduced to single operation **join**

- Write an expression to get a list of all students taking the course ICT252 – the list should show each student's full name.

$$R = \Pi_{\text{stud_firstname, stud_fathersname}}$$
$$(\sigma_{\text{student.stud_id=studentCourse.stud_id}}$$
$$\sigma_{\text{course_code="ICT252"}} (\text{student} \times \text{studentCourse})$$
$$))$$

Example: The table **E** (for **EMPLOYEE**)

enr	ename	dept
1	Bill	A
2	Sarah	C
3	John	A

Example: The table **D** (for **DEPARTMENT**)

dnr	dname
A	Marketing
B	Sales
C	Legal

SQL	Result					Relational algebra
select * from E, D	enr	ename	dept	dnr	dname	E X D
	1	Bill	A	A	Marketing	
	1	Bill	A	B	Sales	
	1	Bill	A	C	Legal	
	2	Sarah	C	A	Marketing	
	2	Sarah	C	B	Sales	
	2	Sarah	C	C	Legal	
	3	John	A	A	Marketing	
	3	John	A	B	Sales	
	3	John	A	C	Legal	

- Seldom useful in practice.

Rename

- $\rho_x(E)$
- To rename the relation given by an expression E to the name x :

Example- Rename

- $\rho_{\text{csdeg_students}} (\sigma_{\text{programme_code}=\text{"CSDEG"}} (\text{student}))$
 - The operation returns the relation given by the expression, and the relation is named csdeg_students. Can now use that name in further operations.
- To find any students named Solomon who are on the CSDEG programme:
 - $\sigma_{\text{stud_firstname}=\text{"Solomon"}} (\rho_{\text{csdeg_students}} (\sigma_{\text{programme_code}=\text{"CSDEG"}} (\text{student})))$
- Or, can simply rename a relation, as a relation is itself a trivial relational algebra expression:
 - $\rho_{\text{student2}} (\text{student})$
- We can also rename the attributes in the relation, using syntax like this: $\rho_x (A1, A2, \dots, A_n) (E)$
 - If the expression E has arity n (n attributes); A1 is a name for the first attribute, A_n is a name for the nth attribute.

Example- Rename

find a course with highest credit hours

- $\Pi_{\text{course.credit_hours}}(\text{course}) -$
- $\Pi_{\text{course.credit_hours}}(\sigma_{\text{course.credit_hours} < c.\text{credit_hours}}(\text{course}$
 $\times \rho_c(\text{course})))$

Additional Operations

- The basic relational algebra operators that we have just looked at are sufficient to express any relational algebra query.
- But even with these, some types of common query that we make on relations become complex and lengthy to express.
- To make some of these easier, there are some additional operations that simplify some common types of query. These are:
 - Set intersection
 - Join
 - Division
 - Assignment

Intersection

$R \cap S$

- Defines relation consisting of set of all tuples in both R and S
- R and S must be union-compatible

Expressed using basic operations:

$$R \cap S = R - (R - S)$$

- $(r - s)$ gives tuples that *are in r but not in s* .
- If we take those tuples away from r , we are left with only tuples that *are in r and in s* .
- It is easier just to use $\cap$.

Example- Intersection

- Write an expression which will find students are taking the course ICT252 and ICT231.

This can be viewed as the intersection of 2 sets or relations:

$$\Pi_{\text{stud_id}} (\sigma_{\text{course_code}=\text{"ICT252"}} (\text{studentCourse}))$$
$$\cap$$
$$\Pi_{\text{stud_id}} (\sigma_{\text{course_code}=\text{"ICT231"}} (\text{studentCourse}))$$

Join Operations

- u Join - derivative of Cartesian product
- u Equivalent to performing Selection, using join predicate as selection formula, over Cartesian product of two operand relations
- u One of most difficult operations to implement efficiently in an RDBMS
- u One reason RDBMSs have intrinsic performance problems

Join Operations

Various forms of join operations

- **Inner Joins:** only those tuples that satisfy the matching criteria are included, while the rest are excluded.
 - Theta join
 - EQUI join(a particular theta join)
 - Natural join
- **Outer join:** along with tuples that satisfy the matching criteria, we also include some or all tuples that do not match the criteria.
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join
 - Semi join

Theta join (θ -join)

$\bowtie_F$

- Defines relation that contains tuples satisfying predicate F from Cartesian product of R and S
- Predicate F is of the form $R.a_i \theta S.b_i$ where θ may be comparison operators ($<$, $\leq$, $>$, $\geq$, $=$, $\neq$)

Theta join (θ -join)

- u Can rewrite Theta join using basic Selection and Cartesian product operations

$$R \bowtie_F S = \sigma_F(R \times S)$$

- u Degree of a Theta join
 - u Sum of degrees of operand relations R and S
 - u If predicate F contains only equality (=)
 - u *Equijoin*

Example - Equijoin

- List the names and comments of all clients who have viewed a property for rent.

$(\Pi_{\text{clientNo}, \text{fName}, \text{lName}} (\text{Client})) \bowtie_{\text{Client.clientNo} = \text{Viewing.clientNo}} (\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}} (\text{Viewing}))$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Natural join

u $R \bowtie S$

- Equijoin of relations R and S over all common attributes x
- One occurrence of each common attribute eliminated from result

Example - Natural join

- List the names and comments of all clients who have viewed a property for rent.

$(\Pi_{\text{clientNo}, \text{fName}, \text{lName}} (\text{Client})) \bowtie$

$(\Pi_{\text{clientNo}, \text{propertyNo}, \text{comment}} (\text{Viewing}))$

clientNo	fName	lName	propertyNo	comment
CR76	John	Kay	PG4	too remote
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR56	Aline	Stewart	PG36	
CR62	Mary	Tregear	PA14	no dining room

Outer join

- u Used to display rows in result that do not have matching values in join column
- u $R \bowtie S$
 - (Left) outer join
 - » Tuples from R that do not have matching values in common columns of S included in result relation

Example - Left Outer join

- u Produce a status report on property viewings.

$\Pi_{\text{propertyNo, street, city}} (\text{PropertyForRent}) \bowtie$
 Viewing

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-01	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-01	no dining room
PL94	6 Argyll St	London	null	null	null
PG4	6 Lawrence St	Glasgow	CR76	20-Apr-01	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-01	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr-01	
PG21	18 Dale Rd	Glasgow	null	null	null
PG16	5 Novar Dr	Glasgow	null	null	null

Semijoin

u $R \bowtie_F S$

- Defines relation that contains tuples of R that participate in join of R with S

u Can rewrite Semijoin using Projection and Join:

$$R \bowtie_F S = \Pi_A(R \bowtie_F S)$$

Example - Semijoin

- List complete details of all staff who work at the branch in Glasgow.

$\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\sigma_{\text{city}='Glasgow'}(\text{Branch}))$

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Division

u $R \div S$

- Defines relation over attributes C that consists of set of tuples from R that match combination of *every* tuple in S

◆ Expressed using basic operations:

$$T_1 \leftarrow \Pi_C(R)$$

$$T_2 \leftarrow \Pi_C((S \times T_1) - R)$$

$$T \leftarrow T_1 - T_2$$

Example - Division

- Identify all clients who have viewed all properties with three rooms.

$$(\Pi_{\text{clientNo}, \text{propertyNo}} (\text{Viewing})) \div (\Pi_{\text{propertyNo}} (\sigma_{\text{rooms} = 3} (\text{PropertyForRent})))$$

$\Pi_{\text{clientNo}, \text{propertyNo}} (\text{Viewing})$		$\Pi_{\text{propertyNo}} (\sigma_{\text{rooms} = 3} (\text{PropertyForRent}))$	RESULT
clientNo	propertyNo	propertyNo	clientNo
CR56	PA14	PG4	CR56
CR76	PG4	PG36	
CR56	PG4		
CR62	PA14		
CR56	PG36		

Aggregate Operations

- u $\mathfrak{I}_{AL}(R)$

- Applies aggregate function list, AL, to R to define relation over aggregate list
- AL contains one or more (`<aggregate_function>`, `<attribute>`) pairs

- u Main aggregate functions:

- COUNT, SUM, AVG, MIN and MAX

Example – Aggregate Operations

- u How many properties cost more than £350 per month to rent?

$\rho_R(\text{myCount}) \mathfrak{I}_{\text{COUNT propertyNo}} (\sigma_{\text{rent} > 350} (\text{PropertyForRent}))$

myCount
5

(a)

Grouping Operation

$\mathfrak{u}_{GA} \mathfrak{F}_{AL}(R)$

- Groups tuples of R by grouping attributes, GA
- Then applies aggregate function list, AL, to define new relation
- AL contains one or more ($\langle \text{aggregate_function} \rangle, \langle \text{attribute} \rangle$) pairs
- Resulting relation contains grouping attributes, GA, along with results of each aggregate function

Example – Grouping Operation

- u Find the number of staff working in each branch and the sum of their salaries.

$\rho_R(\text{branchNo}, \text{myCount}, \text{mySum})$

branchNo $\mathfrak{S}$ COUNT staffNo, SUM salary (Staff)

branchNo	myCount	mySum
B003	3	54000
B005	2	39000
B007	1	9000

Order of Preference

u Precedence of relational operators :

- $[\sigma, \pi, \rho]$ (highest)
- $[X, \bowtie]$
- $\cap$
- $[U, \text{—}]$

Relational Calculus

- u Relational calculus query specifies *what* is to be retrieved rather than *how* to retrieve it
 - No description of how to evaluate a query
- u In first-order logic (or predicate calculus), *predicate* is a truth-valued function with arguments
- u Proposition
 - Substitution of values for arguments in predicate
 - Can be either true or false

Relational Calculus

- u If predicate contains a variable, must be range for x
- u Substitution of some values of range for x , proposition may be true; for other values, false
- u When applied to databases, relational calculus has forms: *tuple* and *domain*

Tuple Relational Calculus

- u Finds tuples for which predicate is true
- u Uses tuple variables
- u Tuple variable
 - Variable that ‘ranges over’ a named relation: i.e., variable whose only permitted values are tuples of the relation
- u Specify range of a tuple variable S as the Staff relation as:
Staff(S)
- u To find set of all tuples S such that $F(S)$ is true:
 $\{S \mid F(S)\}$
where F is a formula

Tuple Relational Calculus - Example

- u To find details of all staff earning more than £10,000:

$\{S \mid \text{Staff}(S) \wedge S.\text{salary} > 10000\}$

- u To find a particular attribute, such as salary, write:

$\{S.\text{salary} \mid \text{Staff}(S) \wedge S.\text{salary} > 10000\}$

Tuple Relational Calculus

- u *Quantifiers* - tell how many instances the predicate applies to:
 - Existential quantifier $\exists$ ('there exists')
 - Universal quantifier $\forall$ ('for all')
- ◆ Tuple variables qualified by $\forall$ or $\exists$ are called *bound* variables, otherwise called *free* variables

Tuple Relational Calculus

- u **Existential quantifier** used in formulae that must be true for at least one instance, such as:

$$\text{Staff}(S) \wedge (\exists B)(\text{Branch}(B) \wedge \\ (\text{B.branchNo} = \text{S.branchNo}) \wedge \text{B.city} = \text{'London'})$$

- u Means ‘There exists a Branch tuple with same branchNo as the branchNo of the current Staff tuple, S , and is located in London’

Tuple Relational Calculus

- u Universal quantifier used in statements about every instance, such as:

$$(\forall B) (B.city \neq \text{'Paris'})$$

- u Means 'For all Branch tuples, the address is not in Paris'

- ◆ Can use $\sim(\exists B) (B.city = \text{'Paris'})$ which means 'There are no branches with an address in Paris'

Tuple Relational Calculus

- u Formulae should be unambiguous and make sense
- u A (well-formed) formula is made out of atoms:
 - » $R(S_i)$, where S_i is a tuple variable and R is a relation
 - » $S_i.a_1 \theta S_j.a_2$
 - » $S_i.a_1 \theta c$
- u Can recursively build up formulae from atoms:
 - » An atom is a formula
 - » If F_1 and F_2 are formulae, so are their conjunction, $F_1 \wedge F_2$; disjunction, $F_1 \vee F_2$; and negation, $\sim F_1$
 - » If F is a formula with free variable X , then $(\exists X)(F)$ and $(\forall X)(F)$ are also formulae

Example - Tuple Relational Calculus

- u List the names of all managers who earn more than £25,000.

$\{S.fName, S.lName \mid Staff(S) \wedge$
 $S.position = 'Manager' \wedge S.salary > 25000\}$

- u List the staff who manage properties for rent in Glasgow.

$\{S \mid Staff(S) \wedge (\exists P) (PropertyForRent(P) \wedge$
 $(P.staffNo = S.staffNo) \wedge P.city = 'Glasgow')\}$

Example - Tuple Relational Calculus

- u List the names of staff who currently do not manage any properties.

$$\{S.fName, S.lName \mid Staff(S) \wedge (\sim(\exists P) \\ (PropertyForRent(P) \wedge (S.staffNo = P.staffNo))))\}$$

Or

$$\{S.fName, S.lName \mid Staff(S) \wedge ((\forall P) \\ (\sim PropertyForRent(P) \vee \\ \sim(S.staffNo = P.staffNo))))\}$$

Example - Tuple Relational Calculus

- u List the names of clients who have viewed a property for rent in Glasgow.

$$\{C.fName, C.lName \mid Client(C) \wedge ((\exists V)(\exists P) \\ (Viewing(V) \wedge PropertyForRent(P) \wedge \\ (C.clientNo = V.clientNo) \wedge \\ (V.propertyNo = P.propertyNo) \wedge \\ P.city = 'Glasgow')))\}$$

Tuple Relational Calculus

- u Expressions can generate infinite set
 - Example
 $\{S \mid \sim \text{Staff}(S)\}$
- u Eliminate by:
 - Adding restriction that all values in result must be values in domain of expression E , $\text{dom}(E)$
- u Domain of E
 - Set of all values that appear explicitly in E or in relations whose names appear in E

Domain Relational Calculus

- ◆ Uses variables that take values from domains

- u A general domain relational calculus expression:

$$\{d_1, d_2, \dots, d_n \mid F(d_1, d_2, \dots, d_n)\}$$

- » $R(d_i)$, where d_i is a domain variable and R is a relation

- » $d_i \theta d_j$

- » $d_i \theta c$

- u Can recursively build up formulae from atoms:

- » An atom is a formula

- » If F_1 and F_2 are formulae, so are their conjunction, $F_1 \wedge F_2$; disjunction, $F_1 \vee F_2$; and negation, $\sim F_1$

- » If F is a formula with free variable X , then $(\exists X)(F)$ and $(\forall X)(F)$ are also formulae

Example - Domain Relational Calculus

- u Find the names of all managers who earn more than £25,000.

$$\{fN, lN \mid (\exists sN, posn, sex, DOB, sal, bN) \\ (Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge \\ posn = 'Manager' \wedge sal > 25000))\}$$

Example - Domain Relational Calculus

- u List the staff who manage properties for rent in Glasgow.

$$\{sN, fN, lN, posn, sex, DOB, sal, bN \mid$$
$$(\exists sN1, cty)(Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge$$
$$PropertyForRent(pN, st, cty, pc, typ, rms,$$
$$rnt, oN, sN1, bN1) \wedge$$
$$(sN=sN1) \wedge cty='Glasgow')\}$$

Example - Domain Relational Calculus

- u List the names of staff who currently do not manage any properties for rent.

$$\{fN, lN \mid (\exists sN) \\ (Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge \\ (\sim(\exists sN1) (PropertyForRent(pN, st, cty, pc, typ, \\ rms, rnt, oN, sN1, bN1) \wedge (sN=sN1))))\}$$

Example - Domain Relational Calculus

- u List the names of clients who have viewed a property for rent in Glasgow.

$$\{fN, lN \mid (\exists cN, cN1, pN, pN1, cty) \\ (Client(cN, fN, lN, tel, pT, mR) \wedge \\ Viewing(cN1, pN1, dt, cmt) \wedge \\ PropertyForRent(pN, st, cty, pc, typ, \\ rms, rnt, oN, sN, bN) \wedge \\ (cN = cN1) \wedge (pN = pN1) \wedge cty = 'Glasgow')\}$$

Domain Relational Calculus

- When restricted to safe expressions, domain relational calculus is equivalent to tuple relational calculus restricted to safe expressions, which is equivalent to relational algebra.
- Means every relational algebra expression has an equivalent relational calculus expression, and vice versa.

Other Languages

- u Transform-oriented languages
 - Non-procedural languages
 - Use relations to transform input data into required outputs (e.g. SQL)

- u Graphical languages
 - Provide user with picture of structure of relation
 - User fills in example of what is wanted and system returns required data in that format (e.g. QBE)

Other Languages

- u 4GLs
 - Create complete customized application
 - Uses limited set of commands in a user-friendly, often menu-driven environment
- u Some systems accept a form of *natural language* , sometimes called a 5GL, although this development is still at an early stage

In-class Questions

Suppliers (sid:integer, sname:string, address:string)

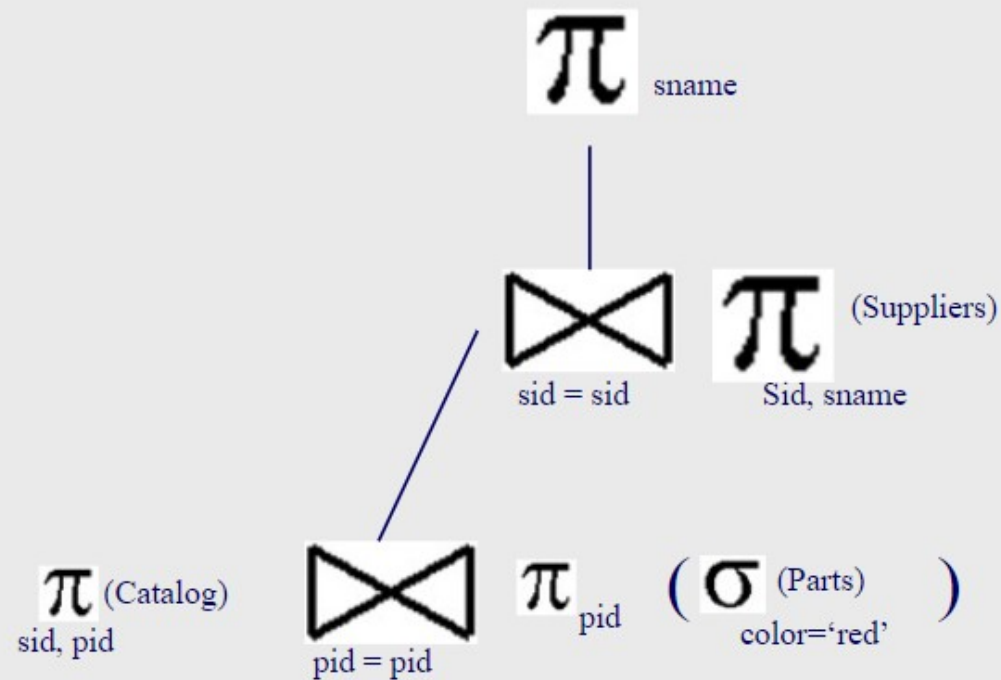
Parts (pid:integer, pname:string, color:string)

Catalog (sid:integer, pid:integer, cost:real)

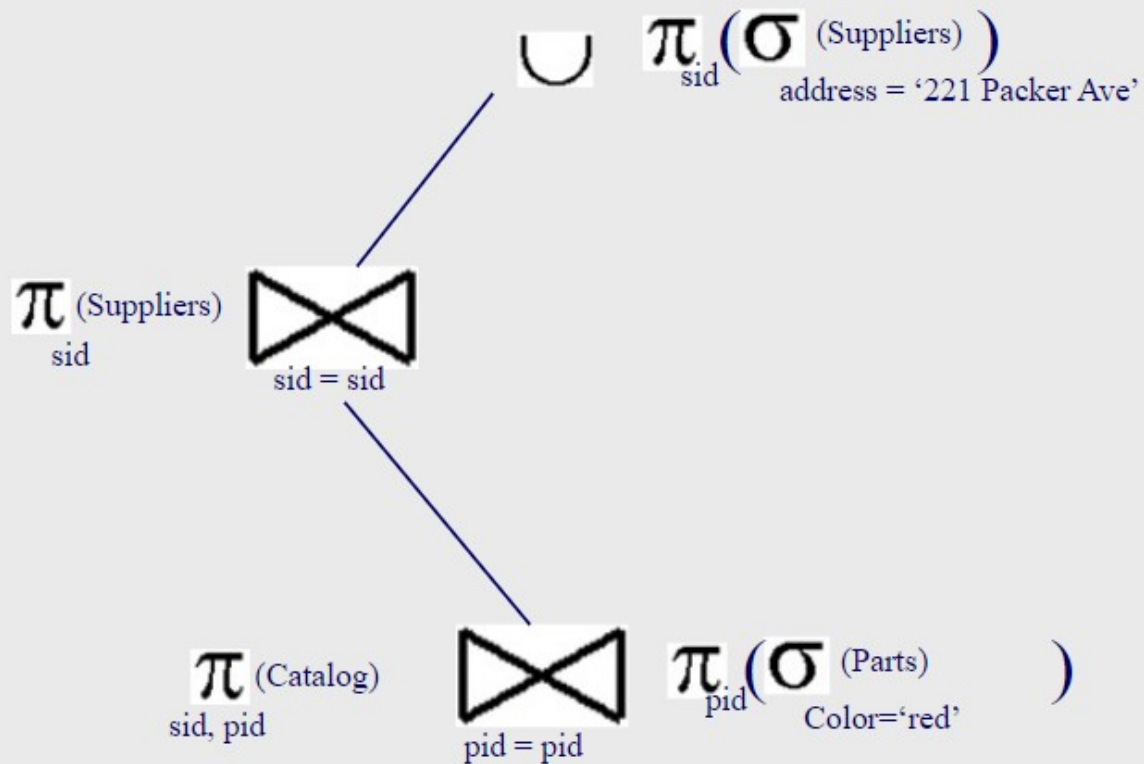
- u Write the following queries in relational algebra and tuple relational calculus:
 - 1. Find the names of suppliers who supply some red part.
 - 2. Find the sids of suppliers who supply some red part or are at 221 Packer Ave.
 - 3. Find the sids of suppliers who supply some red part and some green part.
 - 4. Find the sids of suppliers who supply every red part.

Relational Algebra Solutions

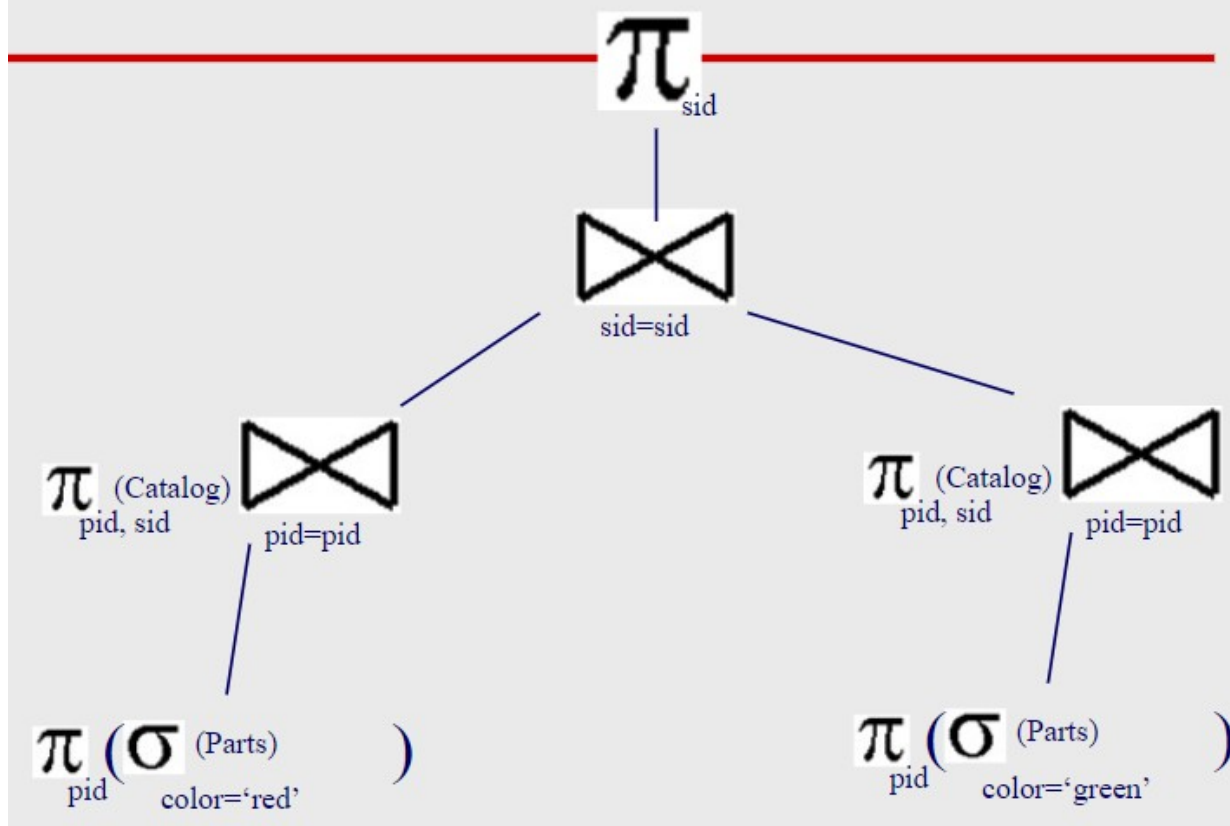
1. Find the names of suppliers who supply some red part.



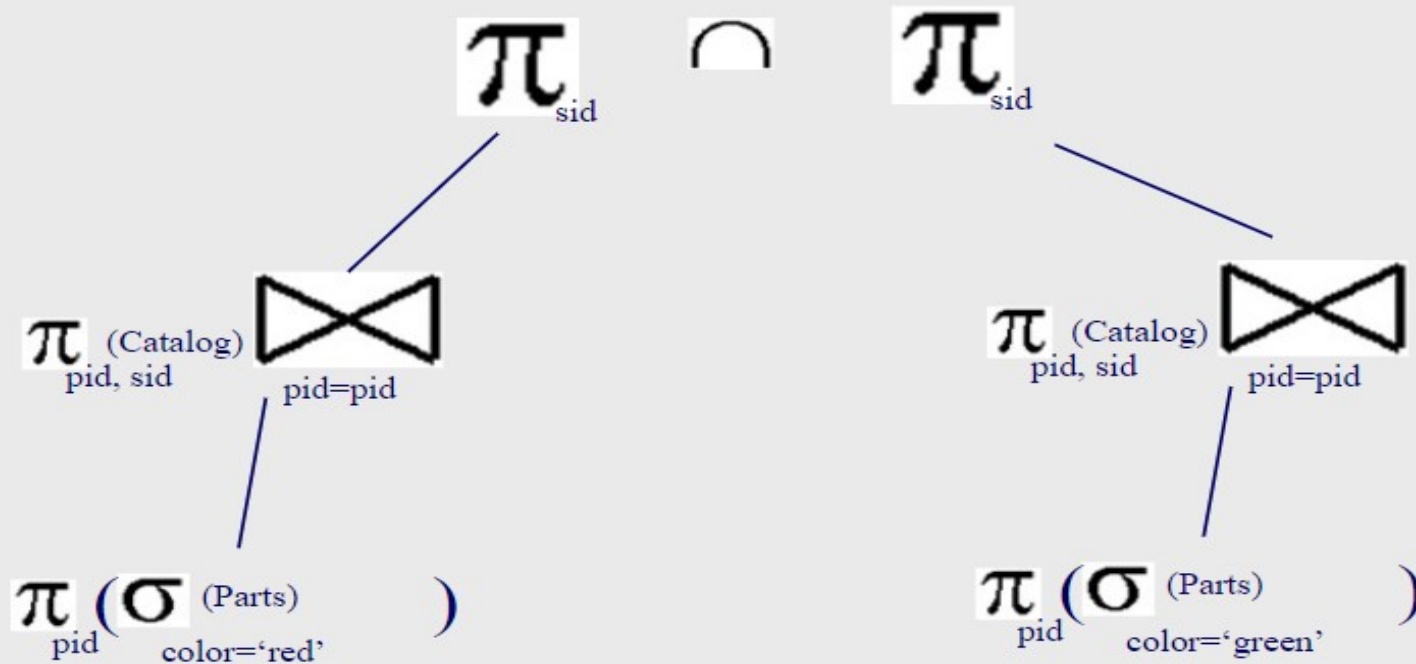
2. Find the sids of suppliers who supply some red part or are at 221 Packer Ave.



3. Find the *sids* of suppliers who supply some red part and some green part



3. Find the *sids* of suppliers who supply some red part and some green part (Alternate Solution).



4. Find the *sids* of suppliers who supply every red part

$$\left(\pi_{\text{pid, sid}}^{\text{(Catalog)}} \right) \supseteq \pi_{\text{pid}} \left(\sigma_{\text{color='red'}}^{\text{(Parts)}} \right)$$

Tuple Relational Calculus Solution

1. $\{S.sname \mid Suppliers(S) \wedge (\exists C)(\exists P)(Catalog(C) \wedge Parts(P) \wedge (C.sid = S.sid) \wedge (P.pid = C.pid) \wedge (P.color = 'red'))\}$
2. $\{S.sid \mid Suppliers(S) \wedge ((S.address = '221 Packer Ave.') \vee ((\exists C)(\exists P)(Catalog(C) \wedge Parts(P) \wedge (S.sid = C.sid) \wedge (C.pid = P.pid) \wedge (P.color = 'red'))))\}$
3. $\{C.sid \mid (Catalog(C) \wedge (\exists P)(Parts(P) \wedge (C.pid = P.pid) \wedge (P.color = 'red')))) \wedge (Catalog(C1) \wedge (\exists P)(Parts(P) \wedge (C1.pid = P.pid) \wedge (P.color = 'green')))) \wedge (C.sid = C1.sid)\}$
4. $\{S.sid \mid Suppliers(S) \wedge ((\forall P)(Parts(P) \wedge (P.color = 'red')))) \wedge ((\forall C)(Catalog(C) \wedge (C.pid = P.pid))) \wedge (C.sid = S.sid)\}$

